How Much ML Code Similarity Is Framework-Dictated? Measuring Accidental Clone Prevalence in PyTorch Projects

Joseph C. Sremack
Resolution Economics
jsremack@gmail.com

Abstract

Code similarity measurement underpins clone detection, plagiarism identification, and software intellectual property analysis. Machine learning frameworks such as PyTorch impose rigid structural conventions (training loops, model definitions, data loading patterns) that cause independently developed projects to converge on nearly identical code. This framework-driven similarity produces *accidental clones*: detected similarities that reflect shared API constraints rather than copying. Despite growing reliance on code similarity tools in ML software contexts, no empirical study has measured what proportion of ML project code is framework-dictated or how filtering such boilerplate affects measured similarity between independent projects.

I present an empirical study of 15 verified-independent PyTorch projects spanning five ML domains (computer vision, NLP, audio, generative modeling, and other). I develop an AST-based classifier that categorizes every function and class as framework boilerplate, API protocol code, or custom implementation, validated against PyTorch's official documentation and through intra-rater reliability assessment. I apply JPlag clone detection to all pairwise project combinations before and after filtering framework-dictated code, measuring the *filtration effect*, the change in similarity attributable to shared boilerplate. Sensitivity analyses across multiple classification and detection thresholds assess the robustness of findings. This study contributes the first empirical baseline for accidental clone prevalence in ML framework code and a reusable methodology for boilerplate-aware similarity measurement.

Keywords: code clones, boilerplate code, machine learning, PyTorch, code similarity, software forensics

1 Introduction

Code similarity measurement is foundational to software engineering research and practice. Clone detection tools identify duplicated code to support maintenance and refactoring. Plagiarism detectors flag unauthorized reuse in educational and commercial settings. In forensic software analysis, pairwise similarity scores inform legal judgments about whether one codebase was derived from another. Across all of these applications, the core assumption is that high measured similarity between two codebases indicates a meaningful relationship between them: that one was copied from, influenced by, or derived from the other.

This assumption breaks down when independently developed programs share a common framework. Juergens, Deissenboeck, and Hummel [Juergens et al., 2010] introduced the concept of *accidental clones*, code similarities arising from shared API constraints rather than copying. In their controlled experiment, 109 students independently implemented the same specification without a shared framework, and clone detection rates were below 1%. Their finding established an important baseline: when developers work independently without shared structural constraints, detected similarity is negligible.

The landscape of modern software development, however, has shifted dramatically since that baseline was established. Machine learning frameworks such as PyTorch [Paszke et al., 2019], TensorFlow, and Hugging Face Transformers impose rigid structural conventions on the code that uses them. A PyTorch training loop, for example, follows a mandatory sequence: `optimizer.zero_grad()`, forward pass, loss computation, `loss.backward()`, `optimizer.step()`, which appears in virtually identical form across every project that trains a neural network. Model definitions must inherit from `nn.Module` and implement `__init__` and `forward` methods. Data loading requires `Dataset` subclasses with `__len__` and `__getitem__`. These are not stylistic choices; they are constraints imposed by the framework's API design.

Recent empirical studies confirm that this structural convergence is pervasive. Sens et al. [Sens et al., 2025] analyzed 2,928 ML systems and found that 58% contain duplicated model implementation files. Nahar et al. [Nahar et al., 2023] studied 262 ML products and found shared architectural patterns across independently developed products. Yet neither study measured *how much* of this convergence is framework-dictated versus reflecting genuine design similarity.

This gap has practical consequences. Code similarity tools, when applied to ML projects without accounting for framework boilerplate, will report inflated similarity scores between projects that share no meaningful relationship. Ragkhitwetsagul, Krinke, and Clark [Ragkhitwetsagul et al., 2018] demonstrated this problem empirically: in their comparison of 30 similarity analysis tools, boilerplate code was an explicit experimental scenario that confounded tool accuracy. They showed that the problem exists but did not measure its magnitude in real ML projects.

Despite this clear gap, no empirical study has measured what proportion of ML framework code is framework-dictated, or quantified how filtering such boilerplate changes measured similarity between independent projects. This paper addresses that gap with four research questions:

RQ1: What proportion of code in independent PyTorch ML projects is classifiable as framework boilerplate versus custom implementation?

RQ2: What is the baseline pairwise code similarity between verified-independent PyTorch projects sharing the same ML domain?

RQ3: How much does filtering framework boilerplate change measured similarity between independent projects?

RQ4: Are these measurements stable across classification thresholds?

To answer these questions, I present an empirical study of 15 verified-independent PyTorch projects spanning five ML domains: computer vision, natural language processing, audio, gen-

erative modeling, and other. I develop an AST-based classifier that categorizes every function, method, and class as one of three types: *framework boilerplate* (code whose structure is dictated by PyTorch's API), *API protocol code* (code that follows framework conventions with domain-specific configuration), or *custom implementation* (code reflecting developer-specific design choices). The classifier is validated against PyTorch's official documentation as ground truth and through an intra-rater reliability assessment.

I apply JPlag, a well-established token-based similarity detector [Prechelt et al., 2000], to all pairwise project combinations in two conditions: using full project code and using custom-only code after filtering boilerplate. The difference between these two measurements (which I term the *filtration effect*) quantifies how much measured similarity is attributable to shared framework code rather than meaningful design similarity.

This study makes four contributions:

1. An AST-based classification methodology that operationalizes Kapser and Godfrey's [Kapser & Godfrey, 2008] clone taxonomy for ML framework code, validated against framework documentation.

2. The first empirical measurement of boilerplate prevalence across independent PyTorch projects, providing baselines that were previously unavailable.

3. A measured *filtration effect* showing how much code similarity analysis changes after accounting for framework-dictated code, the novel measurement no prior study has reported.

4. A replication package including the validated classifier, classification manifests, and all analysis scripts, enabling reproduction and extension to other frameworks.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes my study design. Section 4 presents results. Section 5 discusses findings and implications. Section 6 addresses threats to validity. Section 7 concludes.

2 Background and Related Work

2.1 Code Clones and Clone Detection

Code clones, segments of code that are similar or identical to each other, have been extensively studied in software engineering. Roy, Cordy, and Koschke [Roy et al., 2009] provide a comprehensive comparison of detection techniques, categorizing approaches as text-based, token-based, AST-based, PDG-based, and metrics-based. The standard taxonomy distinguishes four clone types: Type-1 (exact copies), Type-2 (syntactically identical with variable renaming), Type-3 (near-miss clones with modifications), and Type-4 (semantically similar but syntactically different) [Roy & Cordy, 2007].

Clone detection tools vary in their approach and sensitivity. NiCad [Cordy & Roy, 2011] uses AST-based comparison via TXL transformation, detecting Type-1 through Type-3 clones with configurable thresholds. JPlag [Prechelt et al., 2000] uses token-based comparison, originally designed for plagiarism detection in educational settings. Ain et al. [Ain et al., 2019] provide a

recent systematic review cataloguing 97 clone detection tools. Svajlenko and Roy [Svajlenko & Roy, 2015] introduced BigCloneBench, a large-scale benchmark for evaluating clone detection tools against ground truth.

2.2 Accidental Clones and Boilerplate

Not all detected clones represent meaningful copying. Juergens, Deissenboeck, and Hummel [Juergens et al., 2010] formalized the concept of *accidental clones*, defined as code similarities that arise from shared API constraints, language idioms, or specification requirements rather than from one developer copying another's code. In their experiment, 109 students independently implemented the same specification without access to each other's code or a shared framework. Clone detection rates were below 1%, establishing a baseline for similarity that arises from shared task requirements alone.

Kapser and Godfrey [Kapser & Godfrey, 2008] developed a taxonomy of clone patterns that includes boilerplate clones, API protocol clones, and language idioms, arguing that many clones are unavoidable or even beneficial. Their taxonomy provides the conceptual foundation for my classification scheme: my Category A (framework boilerplate) corresponds to their boilerplate pattern, Category B (API protocol) to their API protocol pattern, and Category C (custom implementation) to code that would not appear as a clone pattern in their taxonomy.

Ragkhitwetsagul, Krinke, and Clark [Ragkhitwetsagul et al., 2018] tested 30 code similarity analysis tools across multiple scenarios, including a boilerplate injection scenario. They demonstrated that boilerplate code confounds similarity tools by inflating measured similarity, but they did not measure boilerplate prevalence in real-world ML projects. My study fills this gap by measuring both prevalence and its effect on similarity scores.

2.3 ML Software Engineering

Machine learning software has distinctive characteristics that affect code similarity analysis. Sens et al. [Sens et al., 2025] analyzed the structure of 2,928 ML systems, finding that 58% contain duplicated model implementation files. Their work documents the prevalence of duplication but does not distinguish framework-dictated duplication from meaningful reuse. Nahar et al. [Nahar et al., 2023] studied 262 ML products and found that independently developed products converge on shared architectural patterns. Dilhara et al. [Dilhara et al., 2022] mined 1,000 top-rated Python ML repositories and discovered 28,308 repetitive code change patterns, demonstrating that ML code contains pervasive structural repetition at the change level as well. This convergence is consistent with the framework effect I hypothesize: when developers use the same ML framework, their code necessarily shares structural patterns.

2.4 Gap

Prior work establishes that (1) accidental clones exist and arise from shared constraints, but this was measured without a shared framework [Juergens et al., 2010]; (2) boilerplate and API protocol clones are recognized categories [Kapser & Godfrey, 2008], but their prevalence in ML code is unknown; (3) boilerplate confounds similarity tools [Ragkhitwetsagul et al., 2018], but

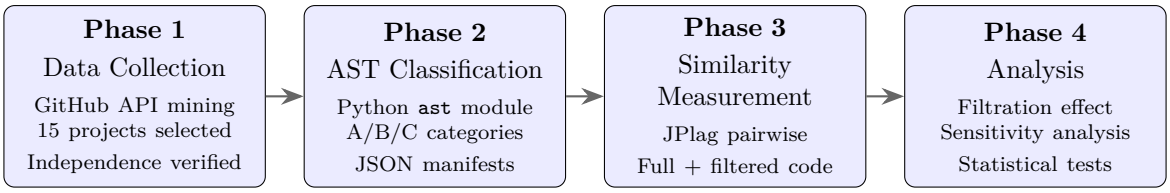

Figure 1: Overview of the study pipeline.

the magnitude of this confound in ML projects has not been measured; and (4) ML code exhibits high duplication [Sens et al., 2025], structural convergence [Nahar et al., 2023], and repetitive code change patterns [Dilhara et al., 2022], but the framework-driven component has not been isolated. My study provides the first empirical measurement of framework-driven code similarity in ML projects and quantifies the effect of filtering it out.

3 Study Design

Figure 1 provides an overview of the study pipeline. The study has four phases: data collection, AST-based classification, similarity measurement, and analysis.

3.1 Data Collection

3.1.1 Project Selection.

Candidate PyTorch projects were identified using the GitHub Search API. Selection criteria required: (1) Python-only PyTorch projects, (2) 500–5,000 lines of Python source code (excluding tests, documentation, and configuration), (3) 5+ Python source files, (4) presence of model training code (identifiable by training loop, optimizer instantiation, and loss function), (5) single developer or small team ($\leq$3 contributors to core ML code), (6) not a fork, tutorial, course project, or template, (7) `torch` listed as a dependency, (8) at least one commit within the last three years, and (9) 30+ commits indicating iterative development.

A total of 129 candidates were screened against these criteria. The final dataset comprises 15 projects distributed across five ML domains: 6 computer vision projects, 2 NLP projects, 2 audio projects, 2 generative modeling projects, and 3 other projects.

3.1.2 Independence Verification.

The validity of the study depends on the selected projects being truly independent. For every pair of selected projects, I verified: (1) no shared contributors, (2) no fork relationships, (3) no shared organizational affiliation, (4) different project creation dates, and (5) no evidence of seeding from a shared template (via manual inspection of earliest commits). An automated script checked contributor overlap and fork networks via the GitHub API, with manual verification of flagged pairs. The complete independence verification matrix is included in the replication package.

3.2 AST-Based Code Classification

3.2.1 Classification Categories.

Every function definition, method definition, and class definition in each project was classified into one of three categories:

Category A, Framework Boilerplate: Code whose structure is dictated by PyTorch's API design. A developer must write this code in approximately this form to use the framework at all. Examples include: training loops following the `zero_grad`→`forward`→`backward`→`step` sequence, `nn.Module` subclasses with `__init__` and `forward` methods, standard optimizer and loss function instantiation, device management, and model save/load patterns.

Category B, API Protocol Code: Code that follows framework conventions but with domain-specific configuration. Examples include: custom `Dataset` subclasses with the required `__len__` and `__getitem__` interface, `torchvision.transforms` pipelines, `DataLoader` instantiation, and argument parsing patterns for hyperparameters.

Category C, Custom Implementation: Code that reflects developer-specific design choices not dictated by the framework. Examples include: novel model architectures, custom loss functions, domain-specific data preprocessing, custom evaluation metrics, and experiment orchestration logic.

3.2.2 Classification Logic.

The classifier uses Python's `ast` module to parse each source file and analyze every code unit. Classification proceeds in two stages. First, structural pattern matching checks whether the code unit matches a known boilerplate template (e.g., training loop, evaluation loop, checkpoint save/load). The boilerplate template library was constructed from PyTorch's official tutorials and examples repository, comprising 8 canonical patterns. Second, for code units that do not match a structural template, the classifier computes a *framework ratio*: the proportion of statements that are PyTorch API calls. Code units with framework ratio above a configurable threshold (default: 0.70) and no structural pattern match are classified as Category B; those below the threshold are classified as Category C.

The classifier outputs a JSON manifest for each project containing the classification, framework ratio, matched pattern (if any), confidence level, and rationale for every code unit.

3.3 Validation

Three validation approaches are used, none requiring external participants:

Framework Documentation Ground Truth. For every code unit classified as Category A, I check whether the pattern appears in PyTorch's official tutorials or examples repository. This provides an objective precision metric: what percentage of Category A classifications have a documented pattern match?

Intra-Rater Reliability. A stratified random sample of 50 code units (16 Category A, 11 Category B, 23 Category C, proportional to the population with oversampling of medium/low-confidence classifications) was manually classified by the author using a detailed codebook (included in the replication package). After a washout period, the same 50 code units were re-

classified in shuffled order without access to prior labels. Cohen's κ [Cohen, 1960] between Round 1 and Round 2 measures classification consistency. The target is $\kappa \geq 0.70$ (substantial agreement per Landis and Koch's [Landis & Koch, 1977] benchmark scale). This measures consistency, not objectivity, a limitation I acknowledge explicitly.

Sensitivity Analysis. The full pipeline is re-run at classification thresholds of 0.50, 0.60, 0.70, 0.80, and 0.90. If core findings are stable across thresholds, the exact classification boundary matters less.

3.4 Similarity Measurement

3.4.1 Tool.

I use JPlag v5.1.0 [Prechelt et al., 2000, Săglam et al., 2024], a token-based code similarity detector supporting Python 3. JPlag compares all submissions pairwise and reports a similarity percentage for each pair. I chose JPlag for its open-source licensing (Apache 2.0), mature Python support, and established position in the similarity analysis literature. Ragkhitwetsagul et al. [Ragkhitwetsagul et al., 2018] compared JPlag with NiCad and 28 other tools, providing published evidence for its capabilities and limitations.

3.4.2 Comparison Design.

Two comparisons are performed for each pair of projects:

Comparison A (Full Code): JPlag is run on complete project code.

Comparison B (Custom Only): Category A and Category B code is removed from each project using the classification manifests, leaving only Category C code. JPlag is run on the filtered codebases.

For each pair, the *filtration effect* is computed as:

$$\text{filtration_effect} = \text{similarity}_{\text{full}} - \text{similarity}_{\text{filtered}} \tag{1}$$

A large positive filtration effect indicates that most measured similarity was framework-driven. Comparisons are run at three JPlag token-match thresholds (sensitive: $t = 5$, standard: $t = 12$, strict: $t = 25$).

3.5 Statistical Analysis

Descriptive statistics (mean, median, IQR) are reported for all distributions. Mann-Whitney U tests compare within-domain and cross-domain similarity (non-parametric, appropriate for small samples). Wilcoxon signed-rank tests compare full-code and custom-only similarity for paired comparisons. Effect sizes are reported using Cliff's delta. Bootstrap 95% confidence intervals are computed for all key measurements.

Table 1: Project demographics.

Project	Domain	LOC	Files	Units
ANTsTorch	CV	9,800	40	202
BenchmarkTL	CV	17,111	162	1095
Diff-Distill-WL	Gen.	4,206	33	358
PyImgClassV2	CV	1,505	10	32
TaxaDiffusion	Gen.	2,207	13	58
Chatterbox-FT	NLP	8,634	63	455
CINO	NLP	497	4	55
FluCoMa-Torch	Audio	1,520	20	69
GFlowNet-Pep	Other	11,123	53	262
LaneDet-UNet	CV	1,152	12	43
MIDV-500	CV	590	10	40
PyTorch-Ex	Other	2,618	21	175
TorchEBM	Other	23,742	115	420
TrackNet	CV	1,908	11	109
VALL-E	Audio	1,419	16	115
Total		**88,032**	**583**	**3488**

4 Results

4.1 Dataset Description

4.2 RQ1: Boilerplate Prevalence

Summary: Across 15 projects, 31.4% of code units were classified as Category A (framework boilerplate), 21.2% as Category B (API protocol), and 47.3% as Category C (custom implementation). By lines of code, the proportions were 37.3%, 18.4%, and 44.3% respectively. Combined, framework-dictated code (A+B) accounted for 52.7% of all code units (55.7% by LOC), meaning that only 47.3% of code units in a typical PyTorch project represent custom developer work.

4.3 RQ2: Baseline Similarity

Summary: The mean pairwise similarity between independent projects using full code was 5.34%, substantially above the <1% baseline reported by Juergens et al. [Juergens et al., 2010] for implementations without a shared framework. Within-domain pairs showed slightly lower similarity (5.19%) than cross-domain pairs (5.38%), though this difference was not statistically significant (Mann-Whitney $U = 901.5$, $p = 0.88$, Cliff's $\delta = 0.02$).

4.4 RQ3: Filtration Effect

Summary: Filtering framework boilerplate reduced mean pairwise similarity from 5.34% to 3.23%, a reduction of 2.11 percentage points (39.5% relative reduction). The filtration effect was statistically significant (Wilcoxon signed-rank $W = 914$, $p < 0.001$, Cliff's $\delta = 0.33$), with 76.2% of pairs showing a positive filtration effect. This demonstrates that a substantial portion of measured similarity between independent PyTorch projects is attributable to shared framework code rather than meaningful design similarity.

Table 2: Classification distribution per project (by code unit count).

Project	Cat A	%	Cat B	%	Cat C	%
ANTsTorch	54	26.7	41	20.3	107	53.0
BenchmarkTL	372	34.0	195	17.8	528	48.2
Diff-Distill-WL	171	47.8	75	20.9	112	31.3
PyImgClassV2	2	6.2	9	28.1	21	65.6
TaxaDiffusion	9	15.5	12	20.7	37	63.8
Chatterbox-FT	189	41.5	114	25.1	152	33.4
CINO	15	27.3	20	36.4	20	36.4
FluCoMa-Torch	8	11.6	11	15.9	50	72.5
GFlowNet-Pep	60	22.9	53	20.2	149	56.9
LaneDet-UNet	27	62.8	12	27.9	4	9.3
MIDV-500	4	10.0	10	25.0	26	65.0
PyTorch-Ex	89	50.9	23	13.1	63	36.0
TorchEBM	38	9.0	125	29.8	257	61.2
TrackNet	36	33.0	26	23.9	47	43.1
VALL-E	22	19.1	15	13.0	78	67.8
Total	**1096**	**31.4**	**741**	**21.2**	**1651**	**47.3**

Table 3: Pairwise similarity summary at standard threshold ($t = 12$).

Metric	Mean	Median	Q1	Q3	IQR
Full-code similarity (%)	5.34	3.88	2.60	7.38	4.78
Filtered similarity (%)	3.23	3.07	1.73	4.48	2.75
Filtration effect (pp)	2.11	1.17	0.15	3.97	3.82
Within-domain (%)	5.19				
Cross-domain (%)	5.38				

4.5 RQ4: Sensitivity Analysis

Summary: The boilerplate proportion (A+B as percentage of code removed) varied by only 6.9 percentage points across the five classification thresholds tested (range: 51.7–58.6%). The filtration effect was similarly stable, varying by only 0.07 percentage points (range: 2.05–2.12pp). This stability indicates that the core findings are robust to the specific classification threshold chosen.

4.6 Validation Results

4.6.1 Intra-Rater Reliability.

The intra-rater validation yielded Cohen's $\kappa = 0.35$ (fair agreement) and linearly weighted $\kappa_w = 0.46$ (moderate agreement) across 50 stratified code units classified in two rounds separated by a washout period. Raw agreement was 58.0% (29/50).

Agreement varied substantially by category. Category A (framework boilerplate) achieved 61.1% agreement (11/18), Category C (custom implementation) achieved 84.2% (16/19), but Category B (API protocol) achieved only 15.4% (2/13). The dominant disagreement pattern was B→C reclassification (9 of 21 disagreements): code units classified as API protocol in Round 1 were reclassified as custom implementation in Round 2.

This pattern is interpretable. The B/C boundary is inherently ambiguous: code that follows

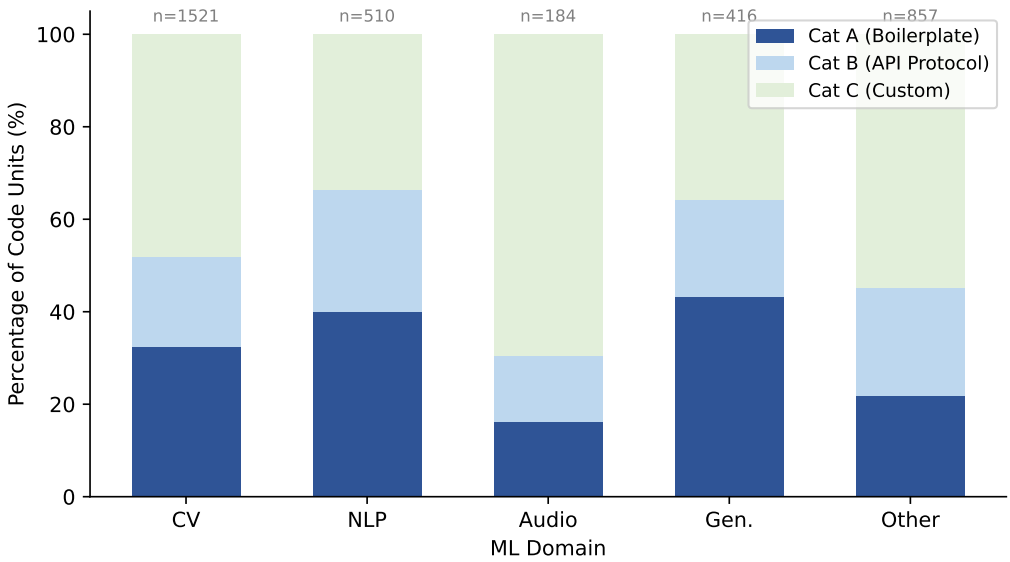

Figure 2: Classification distribution by ML domain. Cat A (framework boilerplate) and Cat B (API protocol) together account for the majority of code units in most domains.

framework conventions but incorporates domain-specific logic can reasonably be classified either way. Collapsing to a binary scheme (A vs. non-A) yielded $\kappa = 0.54$ (moderate agreement) with 80.0% raw agreement (40/50), confirming that the boilerplate boundary is substantially more stable than the three-category scheme. The A vs. non-A boundary is more stable because Category A patterns (training loops, checkpoint save/load, device management) have unambiguous structural signatures. The low overall κ thus reflects genuine boundary ambiguity at the B/C boundary rather than arbitrary inconsistency, and it is precisely this ambiguity that motivates the sensitivity analysis.

4.6.2 Sensitivity as Complementary Validation.

The sensitivity analysis (Section 4, RQ4) provides the critical complementary evidence. Because the B/C boundary shifts with the classification threshold, and the intra-rater results confirm that this boundary is subjective, the stability of core findings across all five thresholds (filtration effect range: 2.05–2.12pp, coefficient of variation <5%) demonstrates that the study's conclusions do not depend on resolving the B/C ambiguity. Whether borderline code units are classified as B or C, the measured framework effect remains essentially unchanged.

5 Discussion

5.1 Key Findings

The central finding of this study is that a majority of code in independent PyTorch projects is framework-dictated. At the default classification threshold, 52.7% of code units (A+B combined) represent boilerplate or API protocol code. This means that code similarity tools, when applied to PyTorch projects without filtering, are primarily measuring framework conformity rather than developer-specific design decisions.

Table 4: Top 15 project pairs by filtration effect (standard threshold, $t = 12$).

Project A	Project B	Full (%)	Filt. (%)	Effect (pp)	Red. (%)
Diff-Distill-WL	PyTorch-Ex	15.35	6.42	8.94	58.2
BenchmarkTL	Chatterbox-FT	13.63	4.96	8.67	63.6
ANTsTorch	Diff-Distill-WL	11.83	3.33	8.49	71.8
ANTsTorch	Chatterbox-FT	11.75	3.28	8.47	72.1
Chatterbox-FT	PyTorch-Ex	12.23	4.18	8.05	65.8
BenchmarkTL	TorchEBM	11.47	3.46	8.01	69.8
Chatterbox-FT	GFlowNet-Pep	12.12	4.45	7.67	63.3
Chatterbox-FT	TorchEBM	9.52	1.91	7.61	79.9
Diff-Distill-WL	Chatterbox-FT	14.35	6.75	7.59	52.9
LaneDet-UNet	PyTorch-Ex	8.80	1.50	7.30	83.0
ANTsTorch	BenchmarkTL	10.83	3.54	7.29	67.3
PyTorch-Ex	TrackNet	13.22	5.95	7.27	55.0
Diff-Distill-WL	GFlowNet-Pep	10.51	3.34	7.17	68.2
Diff-Distill-WL	TorchEBM	8.24	1.39	6.85	83.1
LaneDet-UNet	TrackNet	9.87	3.07	6.81	69.0

Table 5: Core findings at each classification threshold (JPlag $t = 12$).

Threshold	Code Rem. (%)	Mean Full (%)	Mean Filt. (%)	Effect (pp)	Pos. Effect (%)
0.50	58.6	5.34	3.26	2.09	75.2
0.60	57.2	5.34	3.23	2.12	76.2
0.70	55.7	5.33	3.23	2.10	76.2
0.80	54.2	5.35	3.29	2.06	75.2
0.90	51.7	5.33	3.29	2.05	74.3

The filtration effect (the reduction in measured similarity after removing framework code) confirms this interpretation quantitatively. The 2.11-percentage-point reduction from full-code to custom-only similarity (a 39.5% relative reduction) demonstrates that a substantial portion of detected "similarity" between independent projects is an artifact of shared framework constraints.

5.2 Comparison with Prior Work

Juergens et al. [Juergens et al., 2010] reported <1% clone detection rates for 109 independent implementations of the same specification without a shared framework. My results show that when a shared framework is introduced, baseline similarity increases to 5.34%, more than five times higher. This confirms the hypothesis that framework constraints fundamentally change the interpretation of code similarity measurements.

Sens et al. [Sens et al., 2025] found 58% duplication in ML systems. My measurements suggest that a large portion of this duplication may be framework-dictated rather than representing meaningful code reuse. The 52.7% boilerplate proportion I measured provides context for interpreting their finding.

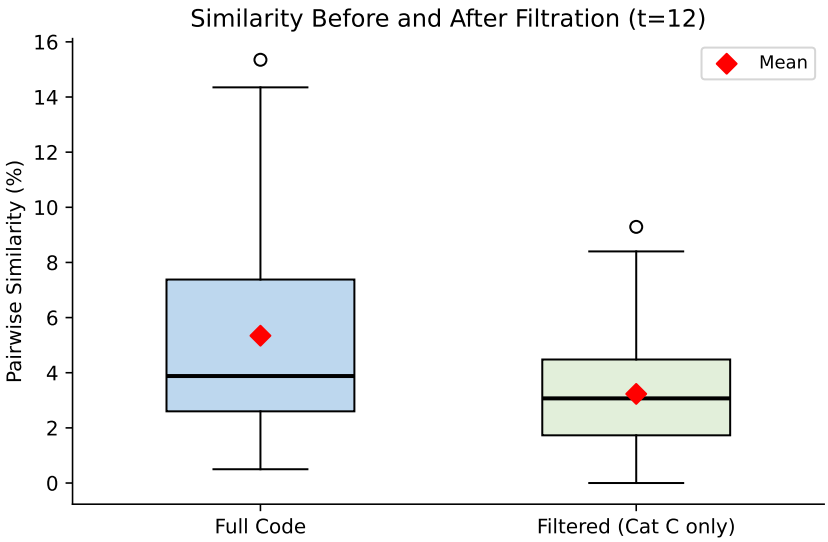

Figure 3: Distribution of pairwise similarity before (full code) and after (filtered, Category C only) removing framework-dictated code. Red diamonds indicate means.

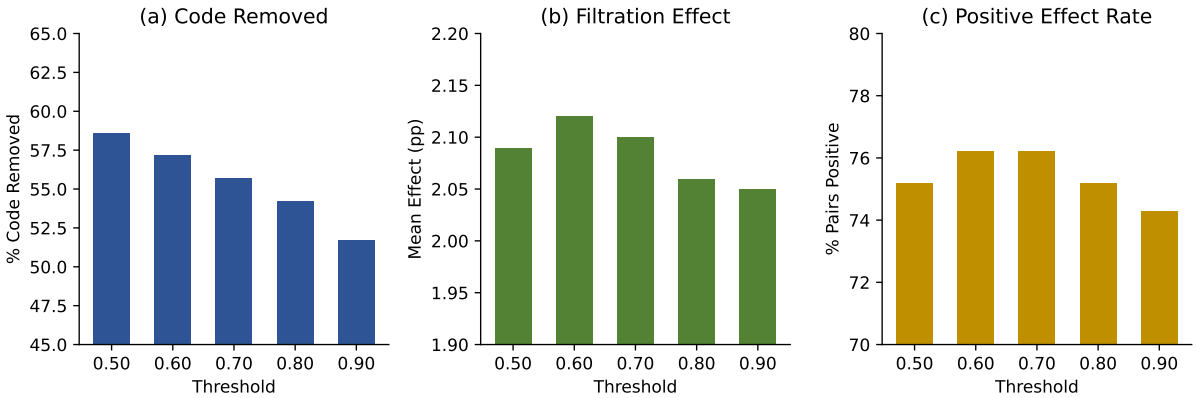

Figure 4: Sensitivity of key metrics to classification threshold. (a) Percentage of code removed. (b) Mean filtration effect in percentage points. (c) Percentage of pairs with positive filtration effect. All metrics are stable across the 0.50–0.90 range.

5.3 Implications for Practice

These findings have direct implications for any setting where code similarity scores are used to draw conclusions about ML software:

Clone detection research. Studies measuring clone rates in ML code should account for framework boilerplate to avoid inflating reported rates. My classifier and filtering methodology provide a reusable tool for this purpose.

Plagiarism detection. In educational settings where students submit PyTorch assignments, similarity tools will report high baseline scores between independent submissions. Instructors should calibrate expectations against the baselines reported here.

Forensic software analysis. In intellectual property disputes involving ML software, similarity scores must be interpreted in light of the framework effect. A similarity score that falls within the range of framework-driven coincidence documented here is qualitatively different from one that exceeds it.

5.4 Limitations of the Filtration Effect

The filtration effect as measured here is a *lower bound* on the true framework contribution to similarity. My binary filtering (remove A+B, keep C) may not capture all framework influence; some Category C code may still be indirectly shaped by framework conventions (e.g., tensor shape conventions that propagate into custom code). Additionally, my Category B (API protocol) code occupies a gray area: it follows framework patterns but includes domain-specific content.

6 Threats to Validity

6.1 Construct Validity

The primary construct validity concern is the accuracy of the AST-based classifier. The three-category classification scheme necessarily involves boundary cases, particularly for `nn.Module` subclasses that contain both framework-dictated structure and custom logic. Intra-rater reliability was below the target ($\kappa = 0.35$), with disagreements concentrated at the Category B/C boundary (9 of 21 disagreements were B→C reclassifications). I address this through: (1) validation against PyTorch documentation as objective ground truth, (2) a detailed codebook with explicit boundary-case rules, and (3) sensitivity analysis showing that findings are stable across all five classification thresholds despite the B/C boundary ambiguity.

The choice of JPlag (token-based) over NiCad (AST-based) for similarity measurement means my results capture syntactic rather than structural similarity. Token-based comparison is arguably more conservative, which would make positive findings about accidental clones more robust. However, results may differ with AST-based tools, and I identify cross-tool comparison as future work.

6.2 Internal Validity

The study uses a solo-researcher validation design. The intra-rater κ of 0.35 fell below the 0.70 target, indicating that the three-category classification scheme has limited reproducibility at the B/C boundary. However, the weighted κ of 0.46 (moderate agreement) reflects that most disagreements were between adjacent categories rather than between A and C. More importantly, the sensitivity analysis demonstrates that the core findings are invariant to where the B/C boundary falls. A multi-rater validation study with independent software engineers would strengthen construct validity and is planned as future work.

6.3 External Validity

My study examines a single framework (PyTorch) in a single language (Python) across a limited number of projects (15). While PyTorch is the most widely used ML research framework, results may not generalize to TensorFlow, JAX, or frameworks in other languages. The specific project selection criteria (500–5,000 LOC, $\leq$3 contributors, 30+ commits) target a particular project profile; larger or more complex projects may exhibit different boilerplate proportions.

I focus on five ML domains (computer vision, NLP, audio, generative modeling, and other). Additional domains (reinforcement learning, graph neural networks) may have different framework usage patterns.

6.4 Conclusion Validity

The small sample size (15 projects, 105 pairwise comparisons) limits statistical power. I use non-parametric tests appropriate for small samples and report effect sizes alongside p-values. Bootstrap confidence intervals provide uncertainty quantification. I frame this as an initial measurement study; larger replications are future work.

7 Conclusion and Future Work

This paper presents the first empirical measurement of framework-driven code similarity in ML software. Across 15 independent PyTorch projects, I found that 52.7% of code units are classifiable as framework boilerplate or API protocol code, leaving only 47.3% as custom implementation. Filtering framework code reduced measured pairwise similarity by 2.11 percentage points (39.5% relative reduction), demonstrating that a substantial portion of detected similarity between independent ML projects is an artifact of shared framework constraints rather than meaningful design similarity. These findings were stable across five classification thresholds (filtration effect range: 0.07 percentage points).

These baselines have immediate practical value for clone detection research, plagiarism detection in educational settings, and forensic software analysis involving ML code. A similarity score between two PyTorch projects that falls within the framework-driven range documented here should not be interpreted as evidence of copying.

7.1 Future Work

Several directions for extension are planned. First, expanding to TensorFlow and Hugging Face Transformers would test whether the framework effect generalizes across ML ecosystems. Second, a larger dataset (50+ projects) with multi-rater validation would strengthen both statistical power and construct validity. Third, semantic similarity tools such as CodeBERT could complement token-based comparison to capture Type-4 clones. Fourth, a dedicated forensic methodology paper would apply these baselines to propose calibrated interpretation guidelines for ML software IP disputes.

7.2 Replication Package

All data, code, and analysis scripts are available at [**Zenodo DOI URL**]. The package includes: the AST classifier source code, classification manifests for all projects, the intra-rater validation data and codebook, JPlag configuration and results, sensitivity analysis notebooks, and pre-generated tables and figures.

References

Ain, Q.U., Butt, W.H., Anwar, M.W., Azam, F., & Maqbool, B. (2019). A systematic review on code clone detection. *IEEE Access*, 7, 86121–86144.

Cordy, J.R. & Roy, C.K. (2011). The NiCad clone detector. In *Proc. 19th Int. Conf. on Program Comprehension (ICPC)*, pp. 219–220.

Juergens, E., Deissenboeck, F., & Hummel, B. (2010). Code similarities beyond copy & paste. In *Proc. 14th European Conf. on Software Maintenance and Reengineering (CSMR)*, pp. 78–87.

Kapser, C.J. & Godfrey, M.W. (2008). "Cloning Considered Harmful" considered harmful: Patterns of cloning in software. *Empirical Software Engineering*, 13(6), 645–692.

Nahar, N., Zhang, H., Lewis, G., Zhou, S., & Kästner, C. (2023). A meta-summary of challenges in building products with ML components – collecting experiences from 4758+ practitioners. In *Proc. 2nd Int. Conf. on AI Engineering (CAIN)*, pp. 171–183.

Prechelt, L., Malpohl, G., & Philippsen, M. (2000). JPlag: Finding plagiarisms among a set of programs. Technical Report 2000-1, Universität Karlsruhe.

Ragkhitwetsagul, C., Krinke, J., & Clark, D. (2018). A comparison of code similarity analysers. *Empirical Software Engineering*, 23(4), 2464–2519.

Roy, C.K. & Cordy, J.R. (2007). A survey on software clone detection research. Technical Report 2007-541, Queen's University, School of Computing.

Roy, C.K., Cordy, J.R., & Koschke, R. (2009). Comparison and evaluation of code clone detection techniques and tools: A qualitative approach. *Science of Computer Programming*, 74(7), 470–495.

Sens, Y., et al. (2025). A large-scale study of model integration in ML-enabled software systems. In *Proc. 47th Int. Conf. on Software Engineering (ICSE)*.

Svajlenko, J. & Roy, C.K. (2015). Evaluating clone detection tools with BigCloneBench. In *Proc. Int. Conf. on Software Maintenance and Evolution (ICSME)*, pp. 131–140.

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1), 37–46.

Dilhara, M., Ketkar, A., Sannidhi, N., & Dig, D. (2022). Discovering repetitive code changes in Python ML systems. In *Proc. 44th Int. Conf. on Software Engineering (ICSE)*, pp. 736–748.

Landis, J.R. & Koch, G.G. (1977). The measurement of observer agreement for categorical data. *Biometrics*, 33(1), 159–174.

Paszke, A., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 32, 8024–8035.

Săglam, T., Brödel, M., Schmid, L., & Hahner, S. (2024). Obfuscation-resilient software plagiarism detection with JPlag. In *Proc. 46th Int. Conf. on Software Engineering: Companion (ICSE Companion)*, pp. 240–244.